NOMAD: In-Network Traffic Obfuscation Against Device Fingerprinting for Industrial Control Systems

Philip Akekudaga
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A
philip.akekudaga@uri.edu

Hui Lin
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A.
huilin@uri.edu

***Abstract*—Device fingerprinting lets adversaries in industrial control systems (ICSs) learn which devices and operations they face before an attack. Existing traffic obfuscation pads, splits and delays packets over encrypted channels. However, ICS networks still carry plain-text protocols such as DNP3, where filler and dummy packets are visible, and ICS fingerprinting relies on the timing between different types of packets, e.g., the cross-layer response time (CLRT) between a TCP acknowledgment and the response. In this paper, we present NOMAD, an in-network traffic obfuscation mechanism that uses the syntax of control units to apply a different strategy to each type of operation on a programmable switch. NOMAD replaces the CLRT of reads with a configured value, holds control commands inside the switch, and splits the control units of every response into one fixed layout, without changing the endpoints or the meaning of any control unit. We implement NOMAD on an Intel Tofino-1 switch and evaluate it with 98,816 DNP3 transactions. The CLRT at the master stays within 10–16 μs of the configured value at the median, and all responses carry 61 bytes and remain conformant to IEEE 1815. Against the CLRT classifier of Formby et al., NOMAD's effect is limited on our testbed: trained before deployment, the classifier falls to chance, as it also does under a constant delay of 1 ms, and retrained on the obfuscated traffic it learns about as much as without NOMAD. All transactions complete within 53 ms without retransmission.**

***Index Terms*—traffic obfuscation**

I. Introduction

Device fingerprinting is an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and design effective attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to initiate physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies present network traffic obfuscation [3], [4]. Device fingerprinting targeting general computing environments typically relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5]. Consequently, existing traffic obfuscation focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Because manipulating communication networks can introduce runtime overhead, recent work has begun offloading traffic obfuscation to programmable network switches, using specialized ASIC to change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods cannot be directly applied to ICS environments for two main reasons. First, existing traffic obfuscation is performed over encrypted channels, which are necessary to mix the obfuscated and normal traffic. ICS networks, on the other hand, still rely on plain-text communications traffic due to a wide range of legacy devices, despite ICS network protocols providing security features [10]. Directly padding or adding dummy packets can easily reveal obfuscated packets, exposing the underlying traffic pattern to adversaries. Second, device fingerprinting methods on ICS devices rely on different features from the ones in general computing environments. Instead of using network layer knowledge, they attempt to use behavior on physical devices to reveal device model or types of operations. For example, work in [11] uses cross-layer response time (CLRT), determined by the latency between the TCP acknowledge packets (ACK) and the actual response from the same device to estimate execution time there. Since each vendor may choose different materials or algorithms to implement the same control functionality, CLRT becomes an effective feature for ICS device fingerprinting.

In this paper, we present NOMAD, an in-network traffic obfuscation mechanism against ICS device fingerprinting, even with plain-text communication. Consequently, adversaries cannot conduct effective reconnaissance using side-channel knowledge to reveal target systems' vulnerabilities (e.g., outdated firmware or vulnerable device models) from network communications. Our solution draws on a key insight: to carry *control units* (e.g., measurements and commands), ICS network protocols map them to numerical indices, with syntax varying across control operations. Instead of applying the same obfuscation strategy to each network packet (as in many existing solutions), we use programmable network switches to

extract operation-related knowledge from network traffic and apply different strategies by splitting, padding, and scheduling control units while maintaining protocol compliance. NOMAD realizes multiple advancements with small runtime overhead: (i) provide flexible obfuscation (e.g., normalize TCP ACK and application response in two different patterns); (ii) preserve communication with legitimate end devices; and (iii) require no reliance on de-obfuscation hardware and encryption over ICS traffic.

We realize NOMAD on a single Intel Tofino-1 programmable switch placed in front of an outstation, and we instantiate three example strategies on the DNP3 protocol. First, the switch holds the TCP acknowledgment and the DNP3 response of a READ or SELECT and releases them at deadlines computed from the request, so that the CLRT a passive observer measures becomes a value the operator configures. Second, it holds each OPERATE command for a configured interval before delivering it, so that the timing of a control operation no longer follows the device. Third, it splits the control units of every response into a fixed layout of index ranges and transport frames, so that all responses have the same size while carrying the same measurements and status. None of these strategies changes the master, the outstation, or the meaning of a control unit. We evaluate NOMAD on a hardware testbed, with OpenDNP3 master and outstation endpoints behind the switch, over 98,816 DNP3 transactions under four obfuscation configurations and eight timing settings. Specifically, NOMAD makes the following contributions:

- **Obfuscates plain-text ICS traffic inside the network.** NOMAD uses the syntax of control units, i.e., function codes, object groups and index ranges, to apply a different strategy to each type of operation on one programmable switch. It needs neither encryption nor a de-obfuscation unit at either end, and the obfuscated traffic remains conformant to IEEE 1815 [10].
- **Replaces fingerprintable timing with configured values.** NOMAD releases the acknowledgment of a transaction at a deadline after the request and the response a configured interval after the acknowledgment, and it holds OPERATE commands before delivery. On hardware, the CLRT measured at the master stays within 10–16 μs of the configured value at the median, and the hold applied to OPERATE does not appear on the master-facing link.
- **Normalizes response sizes while preserving protocol compliance.** NOMAD re-encodes READ responses into three index ranges and splits select-before-operate responses into three transport frames, so that every response carries 61 bytes. All 98,816 responses passed the conformance checker with their application sequence intact.
- **Evaluates NOMAD against CLRT fingerprinting with little overhead.** The CLRT classifier of Formby et al., trained before deployment, drops to chance in every setting, as it also does under a constant delay; retrained after deployment on our software outstation, it stays near its unobfuscated accuracy. Every transaction completes within 53 ms without retransmission. Every request is answered at the master within 53 ms, under the 201 ms TCP retransmission timeout, and no request was retransmitted.

II. Background

A. ICS Network Communications

Like many ICSs, power grids comprise cyber and physical infrastructures. *Physically*, substations (buses) are linked by electrical lines into a transmission or distribution network. The *cyber* infrastructure connects field devices, e.g., IEDs (intelligent electronic devices), PLCs (programmable logic controllers), and legacy sensors and breakers, to a control center over an IP-based *control network*. The control center uses supervisory control and data acquisition (or a similar setup) to periodically collect measurements (voltage, current, power consumption) and issue commands to maintain stable, safe physical processes.

ICS network protocols, e.g., DNP3 and Modbus, are designed mainly to carry control units, e.g., measurements or control commands, to accomplish these functions. The syntax of control units varies across operational types. For example, the DNP3 protocol specifies primitive types, e.g., floating points, to represent measurements and composite types, e.g., control relay output block (CROB), to represent commands applied to circuit breakers. The protocol further uses numerical indices to organize various control units in a single network packet. The protocol-conforming devices, e.g., IEDs and PLCs, store a local configuration file, to look up the meaning of a control unit based on its indices.

For example, Table I lists the analog input points that an SEL protective relay exposes over DNP3. Each measurement, from the three phase currents to the power factor, travels only as an index between 0 and 20, and the master interprets a value through the same configuration. A DNP3 request names the points it wants by an object group, a qualifier, and a range of indices, and the response returns the values under object headers that name the indices they carry [10]. Consequently, the same set of control units admits more than one valid layout, e.g., one range of indices or several consecutive ranges, and a response can change its size without changing what it says (Section V).

B. ICS Device Fingerprinting

Because physical devices may use different algorithms or mechanical components to implement the same control operations, adversaries can use side-channel knowledge from control networks, e.g., packet size or timing, to pinpoint specific device types and control operations. This information can reveal vulnerable device models or firmware versions, enabling effective attacks. For example, studies in [11] use two proprietary features for device fingerprinting, as shown in Fig. 1. Cross-layer response time (CLRT), calculated by the latency between a TCP ACK packet and the corresponding application layer response, estimates the time used by specific IEDs to retrieve measurements. Physical Response Time (PRT), calculated as

TABLE I
DNP3 ANALOG INPUT POINTS OF AN SEL PROTECTIVE RELAY. THE MEANING OF EACH INDEX COMES ONLY FROM THE DEVICE CONFIGURATION.

Index	Point	Index	Point
0	IA_MAG	11	VAVE
1	IB_MAG	12	3V2
2	IC_MAG	13	P
3	IG_MAG	14	Q
4	IN_MAG	15	S
5	IAV	16	PF
6	3I2	17	AI301
7	FREQ	18	AI302
8	VAB_MAG	19	AI303
9	VBC_MAG	20	AI304
10	VCA_MAG		

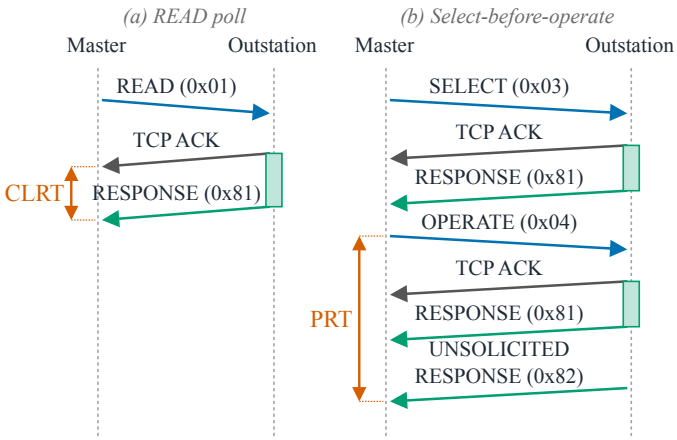


Fig. 1. The two DNP3 transactions on the master-facing link. (a) A READ poll and its CLRT. (b) A select-before-operate control and its PRT.

the latency between unsolicited responses (indicating the start of a control operation) and the corresponding application-layer response, estimates the time specific IEDs take to execute control commands. These device fingerprinting presents different features compared to website or browser fingerprinting in general computing environments. While the latter treat every packet equally (as they are encrypted), device fingerprinting refers to timing between different types of packets to reveal device-specific features.

Fig. 1 shows the two transactions this paper considers and the two intervals that fingerprinting measures on them. In a READ poll, the outstation's TCP stack acknowledges the request as soon as it arrives, while its application layer answers only after the device has retrieved the measurements. CLRT is the interval between the arrival of the TCP acknowledgment and the arrival of the DNP3 response at the master [11]. Because both packets leave the same device over the same path, the network delay cancels, and CLRT estimates how long the device takes to serve the request. In a select-before-operate (SBO) control, the master first sends SELECT and then OPERATE. After the outstation executes the command, it reports the resulting change of state in an UNSOLICITED RESPONSE, and PRT, which Formby et al. call operation

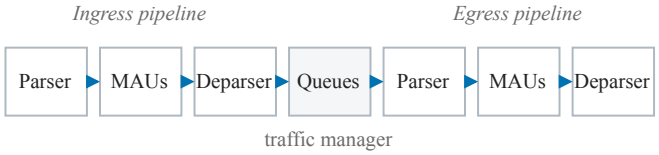


Fig. 2. A programmable data plane. Packets pass a parser, a sequence of match/action units (MAUs) and a deparser in ingress, the traffic manager's queues, and the same three blocks in egress.

time, is the interval from the command to that unsolicited response, which estimates how long the device takes to actuate [11]. In this paper, NOMAD shapes CLRT, and PRT is outside the evaluated scope (Section VII). For example, the SEL-751A relay in our testbed acknowledges a READ poll within 0.49 ms and answers it 2.79 ms later at the median (Section III).

C. Network Switch with Programmable Network Dataplane

Network switches with programmable network data plane enabled by P4 draw significant attention to host in-network security analytics. P4 stands for Programming Protocol-independent Packet Processors and is the de facto data-plane programming language enable customized packet-processing functions [12]. As shown in Fig. 2, Portable Switch Architecture (PSA) provides reference infrastructure for a programmable network data plane. It can feature up to four ingress and four egress pipelines [13], each composed of a parser, a deparser, and a sequence of match/action units (MAUs). MAUs include various storage units (e.g., SRAM, TCAM) to store network knowledge and arithmetic logic units (ALUs) to enable the following features directly in network devices: customized packet parsing across all network layers, line-rate analysis, and integrated monitoring and traffic engineering. However, realizing these benefits requires meeting with strict hardware limits, e.g., roughly 120 MB of SRAM, 10 MB of TCAM, and ALUs that do not support floating-point operations, multiplication, or division.

III. NOMAD Design Overview

In this section, we first present the threat model used by current ICS device fingerprinting. The assumptions about adversaries lead to NOMAD's design principles and major building components.

A. Threat Model

Assumptions on Adversary's Capability. We assume the adversary has penetrated the control network already and can passively monitor traffic from the master-facing link as shown in Figure 3. The passive observer is able to read and record packet headers, their sizes, and arrival time, and because DNP3 does not have any transport security by default [14], this gives the observer the ability to read the function codes of each request, and using that, they are able to label each exchange by what it does, e.g., READ poll or SELECT and OPERATE. We also assume that the adversary knows the existence of the

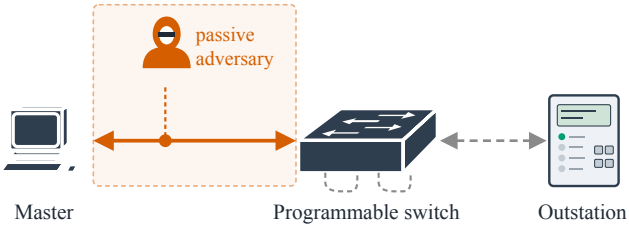


Fig. 3. Observation model. The adversary monitors the shaded master-facing link. The dashed paths are not visible to it.

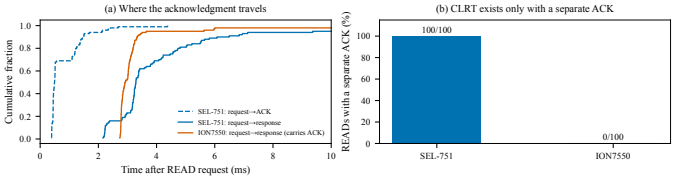


Fig. 4. The same Class 0 READ poll on two IEDs behind the testbed switch. (a) The SEL-751A acknowledges each request in a separate TCP segment, so its CLRT exists; the ION7550 lets the response carry the acknowledgment. (b) Share of READ polls with a separate acknowledgment, 100 polls per device.

framework and the intervals it obfuscates, but they cannot read the internal mechanism and state of the switch or be able to monitor the link between the switch and the outstation. This is practical, as the switch usually sits at the edge of the outstation and is usually connected with a single cable between devices in the same substation cabinet.

Adversary's Goal. The aim of the adversary is to fingerprint the outstation using the timing from the exchanges and then infer the make and model of devices and the operation each of them performs. With this knowledge, they can design specific and targeted attacks like false data injection that must match the devices they compromise [15]. We consider an adversary that builds its classifier from captured traffic before the framework is deployed and applies it afterwards.

B. Design Goals

Before describing NOMAD's design goal, we use a sample network trace collected from IEDs from two different vendors, i.e., the Schweitzer Engineering Laboratories (SEL) 751A [16] and the Schneider Electric (SE) ION7550 power meter [17], to demonstrate unique characteristics in ICS device fingerprinting. First, despite establishing security extensions in common ICS network protocols, plain-text communication is still prevalent in power grids and other ICSs. We have developed a web crawler that examined public product manuals, datasheets, and pages from five IED vendors, namely, SEL, SE, Rockwell Automation, GE Vernova, and Siemens. With a stratified sample of 99 catalogued entries, 32 were identified as having DNP3 support, and these documents established encryption for six entries, and one had DNP3 authentication. However, it is also established that OT asset owners and operators do not use these security measures even if they exist, with numerous reasons hindering their adoption [18]. The crawler methodology and detailed results are presented in Appendix A.

Second, different IEDs can configure network protocols differently, and some even include undocumented variants. These configurations, largely benefiting from plain-text communication, can enable ad hoc features for ICS device fingerprinting. In Figure 4, we present the data acquisition based on the DNP3 protocol on IEDs from two different vendors, i.e., the Schweitzer Engineering Laboratories (SEL) 751A [16] and the Schneider Electric (SE) ION7550 power meter [17]. Based on the network trace, we find that SEL 751A disables TCP's "delayed ACK" mechanism, making CLRT a viable fingerprinting feature. However, SE ION7550 enables this mechanism, so we can no longer use CLRT to estimate how long the device spent retrieving measurements. and configure the DNP3 protocol with different parameters. We believe the proprietary network protocol implementation and configuration can form unique device fingerprints, which also present an excellent opportunity to disrupt adversaries' reconnaissance if a different device fingerprint can be manipulated.

To counter unique ICS device fingerprinting, we design NOMAD driven by the following goals.

- **Obfuscation over Plain-text.** Despite ICS network protocols are gradually enhanced with security components, plain-text communication still prevails in ICS due to existing legacy devices and demanding communication requirements. It is essential to apply various obfuscation strategies, including packet splitting and padding, over plain-text communication without revealing the fact.
- **Flexible Obfuscation.** Features from different types of network packets can combine to form unique fingerprints for IEDs. Applying different obfuscation strategies based on packet types and extracted features is important to accommodate ad-hoc fingerprinting methods.
- **Low Runtime Overhead.** Maintaining constant latency while applying various obfuscation strategies on thousands of IEDs in a power grid is essential to disrupt diverse device fingerprinting.
- **No Reliance on De-obfuscation.** If we follow existing obfuscation solutions, a de-obfuscation unit processes network traffic from obfuscated traffic from all IED, presenting security and overhead bottleneck. It is critical to make obfuscation directly protocol-conformance, putting no reliance on de-obfuscation.

Objectives. To make these goals concrete for the transactions in Fig. 1, NOMAD pursues three research objectives, which the evaluation answers one by one (Section VII).

- **RO1:** For READ, SELECT and OPERATE, the timing a passive observer measures on the master-facing link equals values that the operator configures, independent of how fast the outstation serves the request.
- **RO2:** Every response carries the same number of bytes and the same control units, and remains conformant to IEEE 1815, so that neither the master nor the outstation needs a de-obfuscation unit.

- **RO3:** The added delay stays within the DNP3 and TCP timers of the master, so that no request is retransmitted, and the whole mechanism fits one programmable switch.

Trusted Components. To make these objectives possible, we only add a switch and a control plane to the trusted computing base. The control plane installs the policy to the switch for enforcement. The master or the outstation is not needed, and neither are they modified for the framework to function.

C. Components and Workflows

As shown in Fig. 5, NOMAD includes two components: (i) *Obfuscation infrastructure* that uses P4-enabled network data planes to apply flexible obfuscation strategies and (ii) *obfuscation objectives/constraints* determining security properties in various scenarios while observing ICS communication requirements. When a network packet traverses a gateway network device, we use a customized parser to extract network attributes and control-related knowledge, then categorize packets and organize the carried control units accordingly in the switch's local storage. Using the control-unit syntax, including sizes and indices, we can split, pad, and schedule control units according to different operations while maintaining protocol compliance. While obfuscation infrastructures provide an in-network hardware foundation, we use switches' control plane to realize various obfuscation objectives under constraints for diverse security applications, e.g., neutralize, mimicry communication patterns, and/or maximize differences between obfuscated and original patterns.

IV. Obfuscation Infrastructure

To apply a different strategy to each type of operation, NOMAD must know, for every packet that crosses the switch, which transaction it belongs to and which control units it carries, and it must be able to hold a packet and release it later without a host in the loop. As shown in Fig. 5, packets entering the switch pass three modules: a *classifier*, which extracts control units and binds packets into transactions; a *holder*, which keeps packets until the strategy releases them; and a *rewriter*, which changes the layout of responses while keeping both TCP endpoints consistent. In this section, we describe each module by its function and its state, and we leave the policies they apply to Section V.

A. Classifier

The classifier parses every TCP segment between the master and the outstation down to the DNP3 application layer. It extracts the function code of each request, e.g., READ, SELECT and OPERATE, and the object group, variation, qualifier and index range of each object header (Section II). Based on these fields and the TCP connection, we bind the request, its TCP acknowledgment and its response into one transaction, and we record the arrival time t_0 of the request, from which the strategies of Section V compute their deadlines. Requests leave the classifier unchanged. Consequently, the classification of a packet adds no byte to the request a device receives.

B. Holder

On our switch, the data plane cannot pause a queue, and a packet cannot wait inside a match/action stage. We therefore hold a packet in a queue of the traffic manager and decide its release with *blocker tokens*, i.e., small internal packets that occupy a queue of higher priority and keep the scheduler from serving the held packet. A token re-enters the switch through recirculation after each pass, so the holder checks the clock once per pass without any host involvement. The holder offers three operations on a packet p whose release deadline d_p the strategy sets, possibly after p arrives; e.g., Strategy 1 fixes an acknowledgment's deadline only once the response of its transaction has been seen, and a response's deadline only once the acknowledgment has been released:

- hold(p, d_p): enqueue p in a hold queue Q_L and inject a blocker token b_p, which carries d_p, into a queue Q_H that the scheduler serves with strict priority over Q_L.
- pass(b_p): on each pass of b_p at time τ, recirculate b_p into Q_H if $\tau < d_p$, and drop it otherwise.
- release(p): the scheduler serves Q_L, and therefore p, only once Q_H is empty.

Acknowledgments, responses and OPERATE commands have separate pairs of queues, so a held packet of one class never waits behind the token of another. Each queue pair holds at most one packet at a time, because DNP3 allows one outstanding request per connection and NOMAD assigns one queue pair per class of packet. Let δ denote the duration of one recirculation pass. Because the last token of p is dropped on its first pass at or after d_p, the release time e_p satisfies

$$d_p \;\le\; e_p \;\le\; d_p + \delta, \tag{1}$$

apart from the time the scheduler needs to serve p once Q_H drains and the 512 ns granularity of the switch clock. If the transaction never completes, a fallback deadline of the strategy releases p (Section VI), and a budget of $B = 240{,}000$ passes per held packet bounds the hold in any case. The held packet itself is never copied or regenerated, so its bytes reach the master exactly as the outstation sent them, apart from the changes of the rewriter.

C. Rewriter

The rewriter applies the response layouts of Section V as a response leaves the switch toward the master. Because a re-encoded response is longer than the original, the TCP byte stream seen by the master no longer matches the one the outstation sent. Let Δ denote the number of bytes by which the rewriter has lengthened the responses of a connection so far. To keep both endpoints consistent, we in effect add Δ to the sequence numbers of the outstation's segments toward the master and subtract it from the acknowledgment numbers of the master's segments toward the outstation. The rewriter changes a response only when its mapping state commits the new layout, so a flow without a configured layout is never lengthened without a matching translation, and the TCP checksum of every rewritten frame is updated accordingly.

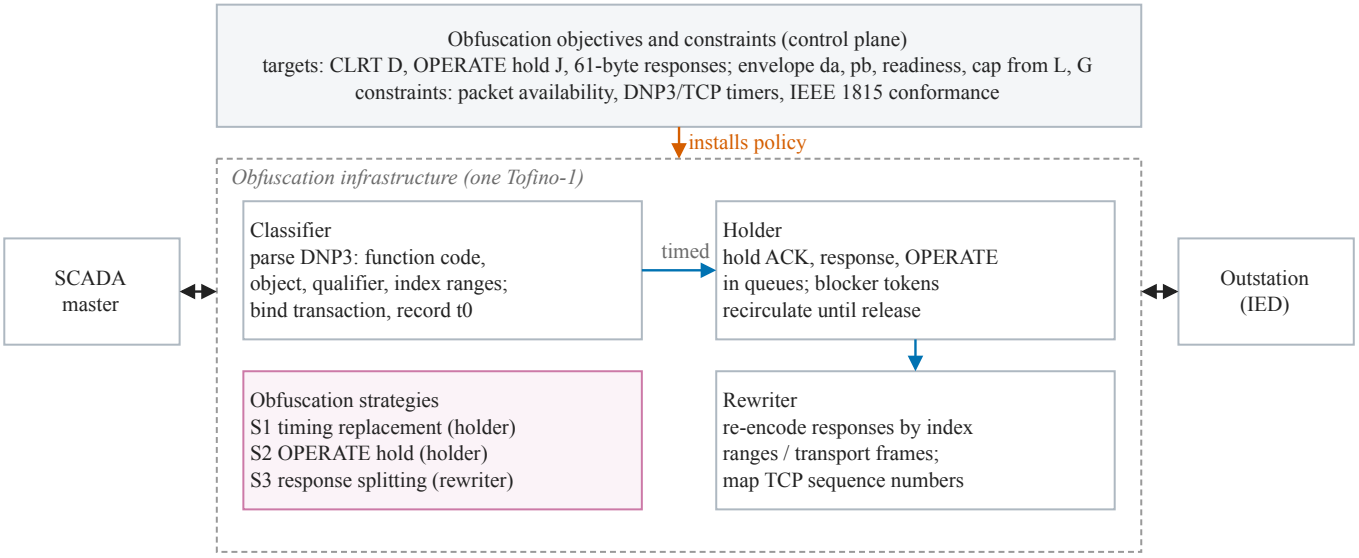


Fig. 5. NOMAD's components. The control plane installs the obfuscation objectives and constraints. On one switch, the classifier parses DNP3 and binds each transaction, the holder keeps acknowledgments, responses and OPERATE commands in queues while blocker tokens recirculate, and the rewriter re-encodes responses and maps TCP sequence numbers.

Control plane. The control plane installs NOMAD's policy into match/action tables at run time: which obfuscation applies to each connection, the configured timing values, and the deadlines of Section VI. All four obfuscation configurations of our evaluation run on the same compiled image and differ only in these entries. Appendix B describes how the three modules map to the switch.

V. Obfuscation Strategies

NOMAD applies a strategy per type of operation rather than per packet. In this section, we present three example strategies, which together cover the timing and size features of the two transactions in Fig. 1: a timing replacement for READ and SELECT, a hold for OPERATE, and a response layout that splits control units. Each strategy is a policy installed by the control plane on the infrastructure of Section IV.

A. Strategy 1: Replacing the Read-Lane Timing

Timing model. In Fig. 6(a), we show the timing of a READ transaction. We denote the arrival time of the request at the switch by t_0, the arrival times of the acknowledgment and the response by t_A and t_R, and the times they depart toward the master by e_A and e_R. Without NOMAD, the master measures

$$\mathrm{CLRT}_{\mathrm{original}} = t_R - t_A, \tag{2}$$

which follows how long the outstation takes to answer.

Replacing the interval. A constant hold on each packet would only shift $\mathrm{CLRT}_{\mathrm{original}}$, and the variation that a timing classifier uses would survive the shift. To remove that variation, we anchor both departures to the request and to the acknowledgment's own release: the switch releases the acknowledgment once a deadline d_a after the request has passed and the response of the same transaction has arrived, and it releases the response a configured interval D after the acknowledgment:

$$e_A = \max(t_0 + d_a,\ t_R), \qquad e_R = e_A + D. \tag{3}$$

Consequently, the interval the master measures becomes

$$e_R - e_A = D, \tag{4}$$

independent of the values of t_A and t_R, as long as the response arrives before the acknowledgment's deadline. A response that arrives early waits longer, and one that arrives close to the deadline waits less. The two holds, $e_A - t_A$ and $e_R - t_R$, are not configured; the schedule determines them. SELECT, the first half of a select-before-operate control, follows the same rule.

Release in practice. Each release is a deadline of the holder, so by Eq. (1) the packet leaves at most one recirculation pass after it, plus the time the scheduler needs to serve it. If the response has not arrived by a later fallback deadline (Section VI), the switch releases what it holds rather than waiting indefinitely.

B. Strategy 2: Holding Control Operations

The OPERATE half of a select-before-operate control carries the physical execution of a command, which Formby et al. fingerprint through PRT [11]. To separate the timing of the operation from its arrival, we hold each OPERATE command at the switch for a configured interval J before it delivers the command to the outstation. The acknowledgment and the response of OPERATE then follow Strategy 1 with the same anchor t_0, the OPERATE's arrival at the switch, as shown in Fig. 6(b). Because the outstation sees the command only after J, its response arrives near $t_0 + J + L$, where L is the time the outstation takes to respond. As long as $d_a \geq J + L$,

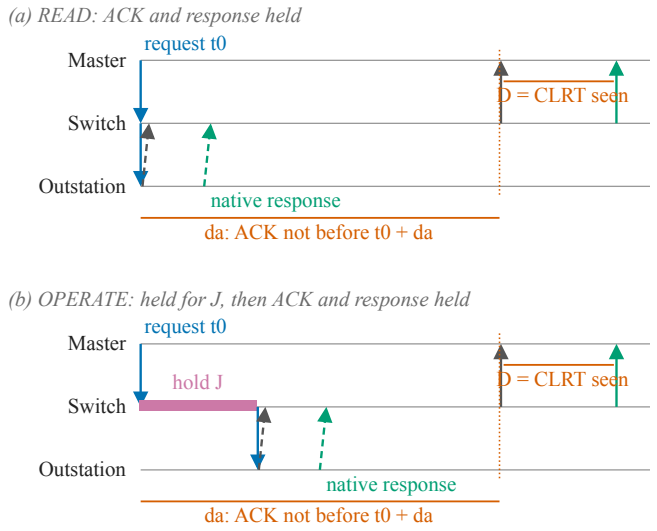


Fig. 6. Release timeline at $D = 10$ ms and $J = 10$ ms, placed at campaign medians. (a) READ: the switch holds the acknowledgment until d_a after the request and releases the response D after it. (b) OPERATE: the switch first holds the command for J, then holds the acknowledgment and the response exactly as in (a).

the acknowledgment still leaves at $t_0 + d_a$ and the response at $e_A + D$, and the master-facing link reveals neither the hold nor the device's execution time. This condition is one of the constraints of Section VI.

C. Strategy 3: Splitting Control Units

Responses of different operations differ in size, and the size alone can reveal the operation. In our testbed, a READ response carries 49 bytes of TCP payload and a SELECT or OPERATE response carries 37 bytes. Padding such a response with filler would either break the DNP3 syntax or expose the filler to an observer who parses the plain text. We, on the other hand, change the layout of the control units the response already carries with the index syntax of Section II.

READ responses. In our testbed, the native READ response returns 23 binary output status points (object group 10, variation 2), indices 0 to 22, in one object header. We re-encode the same points as three consecutive index ranges, 0–5 and 6–9 with one-byte start and stop indices and 10–22 with two-byte indices. Every value appears once, in the same order, and the response grows to 61 bytes.

SELECT and OPERATE responses. The 22 application bytes of the native response are split as 15, 6 and 1 bytes across three DNP3 transport frames of 28, 19 and 14 bytes, which travel in one TCP segment of 61 bytes. The master's DNP3 stack reassembles the three frames into the original application fragment.

In both cases, a normalized response carries 61 bytes, the master receives the same control units in the same order, and every frame keeps valid CRCs and transport sequence numbers. Requests stay as the master sent them; NOMAD normalizes the responses only.

VI. Obfuscation Objectives and Constraints

The strategies of Section V take their parameters from the control plane, and the parameters cannot be chosen freely. An obfuscation objective states what the observer should see; a constraint states what the control network and the switch must still guarantee. In this section, we describe both and the rule NOMAD uses to turn them into deadlines.

Objectives. The operator states three targets: the CLRT the master should observe, D; the hold applied to every OPERATE, J; and the size of every response, 61 bytes in our deployment. Because the targets are values rather than functions of a device, the same targets can apply to all outstations behind the switch that acknowledge requests separately, so that such devices present the same timing and size to the master-facing link.

Packet availability. The switch cannot release a packet it has not received. Strategy 1 therefore requires the response to arrive before the acknowledgment's deadline, $t_R \le t_0 + d_a$, and for OPERATE the response arrives only after the hold, so d_a must also cover J. We denote by L an estimate of the largest time an outstation takes to make its response available after a request. We measure L on unobfuscated pilot traffic, before any hold is applied; in our testbed, L is 5.598 ms.

Deadlines. To keep a margin for queue service and measurement error, we add a guard G of 5 ms and size every deadline for the largest targets the operator may choose, $D_{\max} = J_{\max} = 20$ ms. With $C(x)$ rounding x up to the next millisecond, the control plane sets

$$\begin{aligned} d_a &= C(L + J_{\max} + G), \\ p_b &= C\big(\max(J_{\max} + G,\ d_a + D_{\max} + G)\big), \\ r &= C\big(\max(p_b + G,\ \max(d_a, L + J_{\max}) + D_{\max} + G)\big), \end{aligned} \tag{5}$$

where d_a is the acknowledgment deadline of Strategy 1, p_b bounds the release of a response that carries its own acknowledgment, and r is the readiness deadline after which the switch releases any held packet. For $L = 5.598$ ms, these rules give $d_a = 31$ ms, $p_b = 56$ ms and $r = 61$ ms, and one envelope serves every combination of D and J up to 20 ms. The control plane also checks a cap of $r + G = 66$ ms on the readiness deadline. These guards are margins we chose for the experiments, not proven worst-case bounds.

Communication requirements. A held transaction must still complete before the master gives up on it. With these deadlines, a response leaves the switch about $d_a + D$ after its request, i.e., 51 ms for the largest D, and the guarded budget of the whole envelope is 76 ms. Both stay well below the 500 ms response timeout configured at the master and the 201 ms TCP retransmission timeout we measured on the endpoints, so a held request is not retransmitted. A deployment with slower devices raises L, and therefore d_a, by the same amount.

Protocol compliance. The layouts of Strategy 3 must keep every frame valid under IEEE 1815, i.e., correct CRCs, transport sequence numbers and application sequence numbers, and must carry the same control units in the same order. Program

E therefore rewrites a response only when its shape and every native CRC match the layout configured for its operation.

VII. Evaluation

We evaluate NOMAD against the three research objectives of Section III. For RO1, we measure whether the timing that the master observes equals the configured values; for RO2, we measure response sizes and protocol conformance; and for RO3, we measure the delay and the switch resources NOMAD needs. Because the objectives exist to defeat fingerprinting, we also measure what the classifier of Formby et al. [11] learns from the obfuscated traffic.

A. Testbed and Methodology

Testbed. We run NOMAD on an Intel Tofino-1 switch with SDE 9.13.2, as three P4 programs (Appendix B). The DNP3 master and outstation are OpenDNP3 [19] endpoints in two network namespaces of one server, connected through the same switch port, so every packet between them crosses the switch, and the outstation answers a request after a fixed 5 ms application delay. We capture every packet on the server's switch-facing interface with microsecond timestamps, once as the endpoints send it and once as the switch delivers it; the master-facing observation of Section III is the delivered stream.

Configurations. We compare four configurations on the same compiled image: *No obfuscation*, *Timing* (Strategies 1 and 2), *Size* (Strategy 3) and *Timing+Size* (all three). We vary the configured CLRT D over 5, 10, 15 and 20 ms with J = 2.4 ms, and the OPERATE hold J over 5, 10, 15 and 20 ms with D = 10 ms, i.e., eight settings, all under the one envelope of Section VI.

Workload and campaign. Each experiment block runs every configuration once, in an order that rotates between blocks, with 65 READ polls and 64 select-before-operate cycles each; we count the SELECT and the OPERATE of a cycle as two transactions. The campaign ran 154 blocks without failure, including pilot and feasibility blocks, and its 128 main blocks produced 98,816 DNP3 transactions, 16 blocks per setting. The first 8 blocks of each setting train the attackers and the last 8 test them, so no tested transaction was seen in training.

Attacker. We use the classifier of Formby et al. [11], a multinomial naive Bayes over 200-bin histograms of CLRT, and apply it to operations rather than device types. Its histograms have a last bin that collects every value above a threshold H, which we set to 1.05 times the largest training CLRT. Because the adversary reads function codes (Section III), it groups the CLRT values of one operation into windows of K = 8, 32 and 64 transactions and classifies each window as READ, SELECT or OPERATE. Following our threat model, the attacker is trained on unobfuscated traffic captured before NOMAD is deployed and is applied afterwards. To bound what it can learn once NOMAD is in place, we also retrain the same classifier on the traffic of each configuration. In both cases, the classifier is trained on the first 8 blocks of a setting and tested on the last 8. We report balanced accuracy, for which chance among three operations is 1/3, and, for the reduction from No obfuscation to Timing+Size at D = 10 ms and J = 2.4 ms, a 95% interval from 10,000 paired bootstrap resamples of the 8 test blocks.

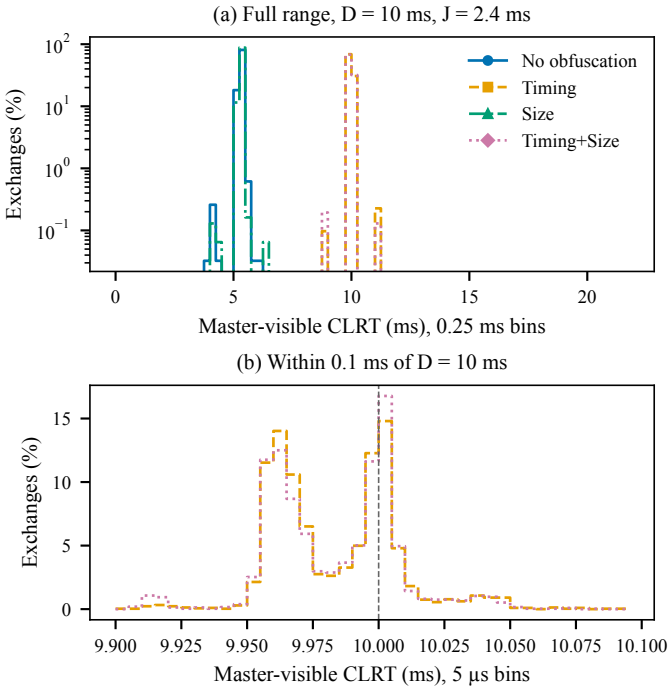


Fig. 7. CLRT measured at the master at D = 10 ms, J = 2.4 ms, (a) over the full range on 0.25 ms bins and (b) within 0.1 ms of D on 5 μs bins. Without timing, CLRT stays at the device's 5.3 ms; with timing, it moves to D.

B. RO1: Timing Replaced by Configured Values

CLRT follows the configuration. In Fig. 7, we show the CLRT the master measures in every configuration at D = 10 ms. Without Strategy 1, CLRT stays at the 5.3 ms the outstation needs, and response-size normalization alone leaves it there. With Strategy 1, the distribution moves to the configured 10 ms and stays within 0.1 ms of it, in two modes about 40 μs apart. Fig. 9 quantifies the error over all eight settings: the median of $|\text{CLRT} - D|$ is 10–16 μs for every operation, the 99th percentile is 58–86 μs, and 0.14–0.46% of the transactions exceed 100 μs. The few exceptions form a tail near 1 ms, with a maximum of 1.93 ms. On an earlier build, we localized this tail to a host-observed delay downstream of the switch's egress parser, and we did not re-measure that attribution on this build.

A small operation-dependent offset. Fig. 8 breaks the error down by setting. At D = 5, 15 and 20 ms, the three operations agree within 3 μs, at about 0, +7 and +11 μs. At D = 10 ms, in all five settings of J, the medians of READ and SELECT sit 20–29 μs early while OPERATE stays within 5 μs of D; because the timed distribution has two modes (Fig. 7), the offset may reflect how often each operation falls in the earlier mode. We have not established the cause of this offset, which

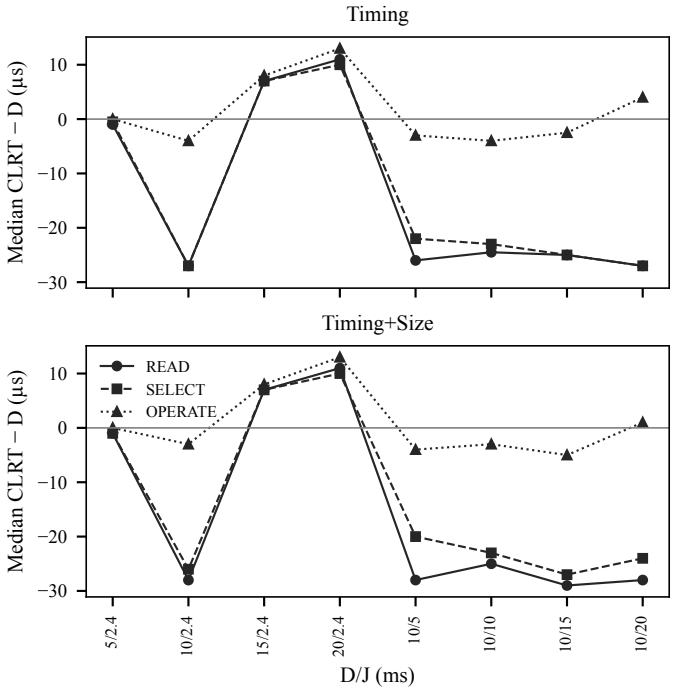


Fig. 8. Median signed error CLRT − D per setting (D/J in ms) and operation, for Timing (top) and Timing+Size (bottom).

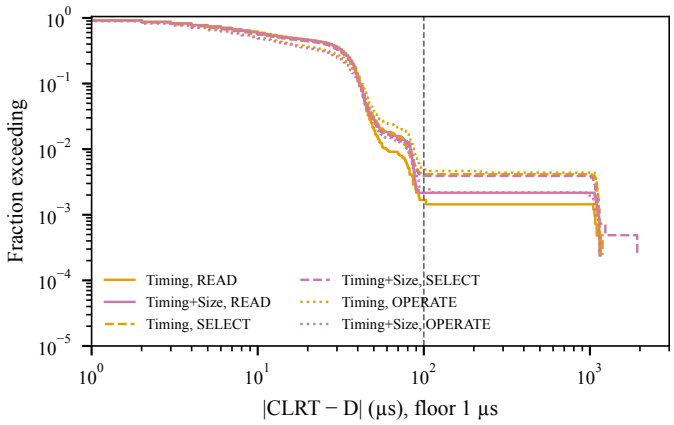


Fig. 9. Complementary CDF of |CLRT − D| for the timed configurations over all eight settings, per operation; the dashed line marks 100 μs.

is far below the 5.3 ms CLRT of the outstation, but it is the one timing difference that remains (Section VII-D).

Replacement rather than shift. A constant hold would keep the device's distribution and only move it. To show the difference, we shifted the unobfuscated CLRT at the primary setting by one constant onto the obfuscated median: the shifted OPERATE stays about 60 μs below READ and SELECT, while the measured Timing+Size distributions of the three operations lie within 25 μs of each other. The median variance of CLRT in held-out windows of 32 transactions is about three times lower with timing than without it. The native spread of our software outstation is already small, with a standard deviation of 0.06–0.11 ms, so the reduction is modest in

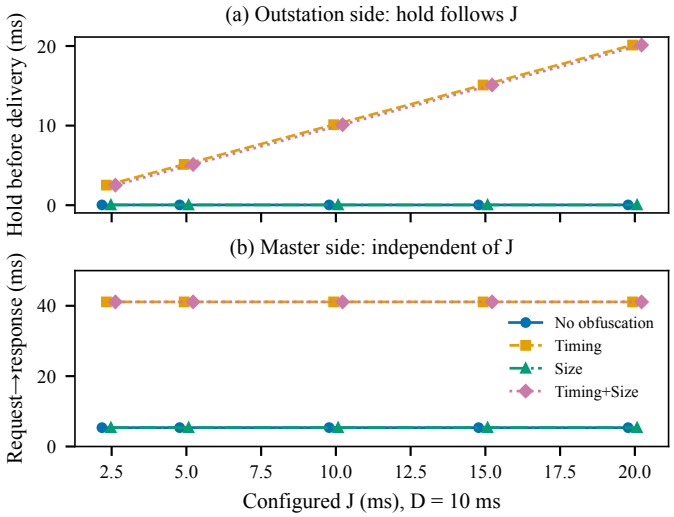


Fig. 10. OPERATE at D = 10 ms for each J: (a) the hold applied before delivery to the outstation and (b) the request-to-response time at the master.

TABLE II
RESPONSES PER CONFIGURATION OVER THE MAIN CAMPAIGN: COUNT, RESPONSES THAT PASSED THE CONFORMANCE CHECKER, RESPONSE SIZES IN BYTES, ACKNOWLEDGMENT COMMITS, FALLBACK RELEASES, AND RETRANSMITTED REQUESTS.

Arm	Resp.	Conform.	Bytes	Commits	Fallb.	Retx.
No obfuscation	24,704	24,704	37/49	0	0	0
Timing	24,704	24,704	37/49	24,704	0	0
Size	24,704	24,704	61	0	0	0
Timing+Size	24,704	24,704	61	24,704	0	0

absolute terms.

The OPERATE hold stays inside the switch. Fig. 10(a) shows that the hold measured before delivery follows the configured J in every setting. Fig. 10(b) shows that the master nevertheless sees the same request-to-response time of 41.1 ms for every J, because d_a covers the hold (Section V). Consequently, the time a device spends on a command no longer reaches the master-facing link.

C. RO2: Uniform and Conformant Responses

Table II summarizes every response of the campaign. Without Strategy 3, READ responses carry 49 bytes and SELECT and OPERATE responses 37 bytes; with it, every response carries 61 bytes, and requests keep their native 20 and 35 bytes. All 98,816 responses passed the IEEE 1815 conformance checker of our harness with intact transport and application sequence numbers, and no DNP3 CRC failed. In each size-normalized configuration, 16,384 select-before-operate responses travelled as three transport frames, and the master reassembled every one of them; every READ matched the outstation's values, and every SELECT authorized its OPERATE, without any packet lost, duplicated or reordered.

D. Resistance to Fingerprinting

Attacker trained before deployment. Fig. 11(a) shows the attacker of our threat model. Without obfuscation, it identifies

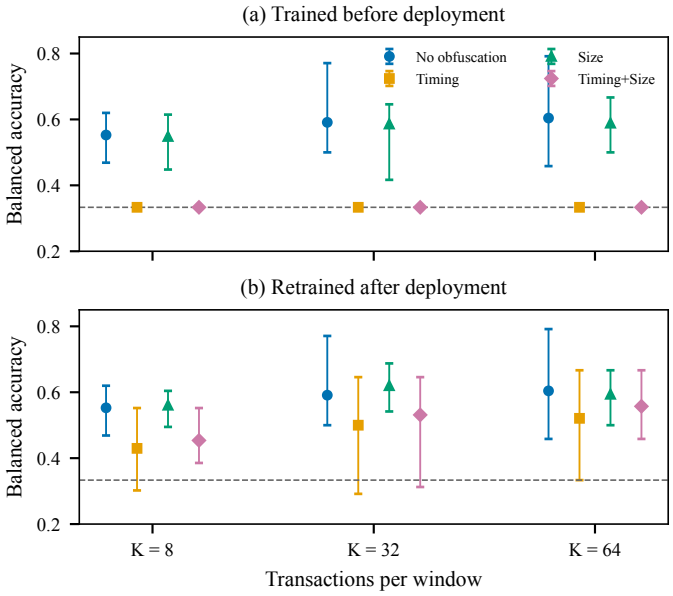


Fig. 11. Balanced accuracy of the classifier of Formby et al. per configuration and window size K, (a) trained before deployment and (b) retrained after deployment. Markers give the mean over the eight settings, bars the range, and the dashed line is chance.

the operation of a window with a balanced accuracy of 0.55–0.60 on average over the eight settings, and Size alone leaves it near that level, because response sizes do not change CLRT. Under Timing and Timing+Size, it falls to 0.333 at every K and in every setting, because every obfuscated window lies outside the histograms it learned from the device. At the primary setting, the reduction is 0.271 at $K = 8$ and 32 and 0.292 at $K = 64$, with 95% intervals of [0.135, 0.396], [0.146, 0.396] and [0.167, 0.417]. However, a constant delay produces the same result. In a post hoc check, shifting the unobfuscated CLRT by 1 ms, or onto the median of Timing+Size, drives the same classifier to 0.33–0.36 on average over the settings. This attacker therefore fails on any timing it did not see in training, and its drop does not show that NOMAD removes operation information; the retrained attacker below is the meaningful test.

Attacker retrained after deployment. Fig. 11(b) shows the same classifier retrained on the traffic of each configuration. Under Timing+Size, it averages 0.454, 0.531 and 0.557 at $K = 8$, 32 and 64, against 0.553, 0.591 and 0.604 without obfuscation, and at the primary setting the reductions are 0.130, 0.000 and −0.042, with intervals that include zero. Consequently, an attacker that collects labeled CLRT after NOMAD is deployed learns about as much as it learned without NOMAD in our testbed. We believe two factors explain this result. First, the unobfuscated CLRT of our software outstation barely differs between operations (Fig. 7), so its histograms carry little operation information to begin with. Second, the 20–29 μs offset at $D = 10$ ms (Fig. 8) leaves a difference that a retrained histogram can learn. Each estimate rests on 192, 48 and 24 test windows per setting at $K = 8$, 32 and 64.

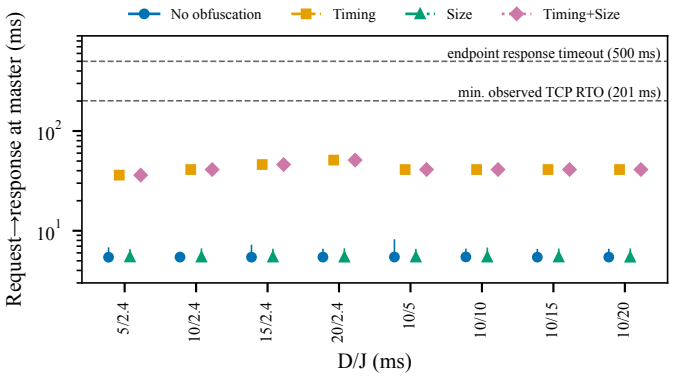


Fig. 12. Request-to-response time at the master per setting (D/J in ms), median (marker) to maximum (line), with the endpoint timers.

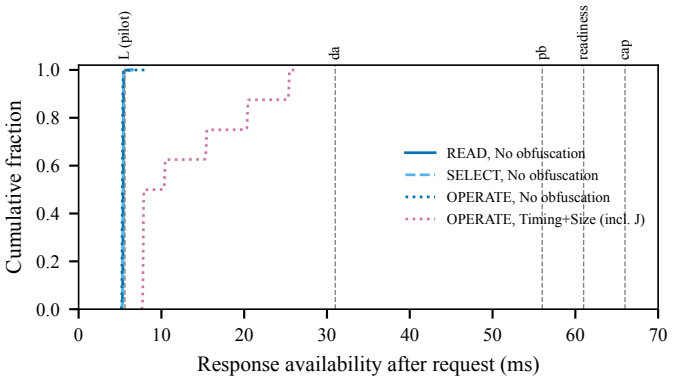


Fig. 13. Response availability after the request at the outstation side, against the envelope of Section VI.

E. RO3: Overhead

Delay. In Fig. 12, we show the request-to-response time at the master. Without timing, a transaction completes in 5.4 ms at the median, and with timing it completes at $d_a + D$, i.e., 36.1–51.1 ms at the median, while the slowest of all 49,408 timed transactions took 52.3 ms. Every transaction completed below the 201 ms TCP retransmission timeout and the 500 ms response timeout of the master, and no request was retransmitted (Table II).

Envelope coverage. Fig. 13 compares the time each response became available at the outstation side with the envelope. Unobfuscated responses were available within 8.0 ms and held OPERATE responses within 26.45 ms, and no response of any configuration arrived after $d_a = 31$ ms. Consequently, every timed acknowledgment committed at its deadline, and no fallback release occurred (Table II); about one blocker token for every two timed transactions outlived its packet and was retired, and delivery was unaffected. The largest availability exceeded the pilot estimate $L = 5.598$ ms, which the guard G absorbed.

Switch resources. NOMAD fits one Tofino-1 switch, and its three programs leave most of the memory of each pipe free (Appendix B).

F. Limitations

Our endpoints are OpenDNP3 software processes behind one switch, and the outstation answers after a fixed delay; physical relays answer with a wider timing spread, which Strategy 1 should replace the same way but which we did not measure on this build. The results are transaction-class identification by the classifier of Formby et al., which uses CLRT, not device identification. Because the adversary reads function codes, the operation of a window is already visible; the classification measures how much operation information CLRT carries, which a device fingerprint built on the same histograms would use, not information the adversary otherwise lacks. Response sizes and device-specific content such as object groups, point counts and status bits remain readable in plain text. With one outstation of nearly constant service time, RO1's independence from how fast a device serves a request follows from the design (Section V), and we did not test it across devices. A device that piggybacks its acknowledgment, like the ION7550, would follow the p_b path, and its master-visible timing would differ from that of a device that acknowledges separately. Our outstation sends no unsolicited responses, so we did not measure PRT, and the OPERATE hold was evaluated only on the acknowledgment and response it shapes. Holding OPERATE delays the physical execution of a command by J, which an operator must weigh against the timing requirements of protection functions. NOMAD leaves requests native, and their sizes alone separate READ from select-before-operate. The cause of the operation-dependent offset at $D = 10$ ms is not established, and the guards of the envelope are margins chosen for the experiments rather than proven bounds.

VIII. Related Work

Device fingerprinting. Devices can be identified by what they emit and how they communicate, even without access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock in TCP timestamps [20], and GTID identifies a device and its type from the timing signature of its packets [21]. Website fingerprinting reaches the same goal from packet sizes, directions and timing over encrypted channels, with attackers based on random forests [22], deep learning [5], and timing alone [23]. NOMAD considers the same passive observer, and its objective is to remove the timing and size features that such attackers rely on from the master-facing link.

ICS and power-system fingerprinting. Formby et al. introduce fingerprinting to control systems and define the cross-layer response time and the physical operation time of a DNP3 outstation [11], and their measurements of substation traffic show how regular these timings are [24]. Later work fingerprints ICS devices from inter-layer response processing time [25], from the scan cycle of PLCs [26], from time delays in industrial dataflows [27], from device physics [28], and from behavior under challenge and response [29], and Keliris and Maniatakos identify Modbus field devices from device-specific protocol content [30], which NOMAD's strategies leave readable.

Such knowledge lets an adversary craft attacks that must match the target, e.g., false data injection against state estimation [15] and attacks on process control [31]. Defenses against reconnaissance in power grids have so far worked on content and connectivity. DefRec virtualizes physical functions and presents deceptive measurements [32], RAINCOAT randomizes data acquisition and injects decoy measurements [33], and a testbed evaluates such approaches on grid infrastructure [34]. Compared to these defenses, NOMAD serves a different objective: it leaves the content of every control unit unchanged and changes when and in which layout the outstation's answer reaches the master.

General traffic obfuscation. Traffic morphing transforms one packet-size distribution into another [3], and Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet sizes and rates at a bandwidth cost [6], WTF-PAD fills inter-packet gaps adaptively [35], Walkie-Talkie molds bursts into supersequences [36], TrafficSliver splits traffic across paths [37], and FRONT, Surakav and RegulaTor trade bandwidth and latency in different ways [38]–[40].

Closer to NOMAD's setting, Handley et al. normalize ambiguous traffic at a network element [41], Smart et al. rewrite TCP/IP behavior to defeat stack fingerprinting [42], and Askarov et al. mitigate timing channels by releasing outputs on a predicted schedule [43], the idea behind NOMAD's deadline release. Shaping also protects smart-home devices [44], and Pacer and NetShaper bound network side-channel leakage in the cloud with shaped schedules [45], [46]. These defenses are effective over encrypted channels with cooperating end hosts that can add cover traffic. However, in a plain-text ICS network, dummy packets and filler bytes are visible to an observer who parses the protocol, and the outstation cannot be modified. NOMAD, on the other hand, changes only the timing and the layout of packets the outstation already sends.

Programmable data planes. P4 defines the programmable parser and match/action model that NOMAD uses [12]. PIFO provides programmable scheduling at line rate [47], SP-PIFO approximates it with strict-priority queues [48], and programmable calendar queues release packets by time on Tofino, recirculating a marker packet whose arrival shows that a queue has drained [49]; NOMAD's blocker tokens play a similar role for a single held packet.

On programmable switches, NetHide obfuscates the observable topology [50], Ditto shapes WAN traffic to a fixed pattern with a switch at each end [7], and Minos and Securitas obfuscate encrypted traffic with padding, dummy packets and fragmentation [8], [9]. In ICS networks, programmable switches enhance protocol security [51] and correlate timed events with deep packet inspection [52]. Compared to this work, NOMAD runs on one switch at the outstation edge and needs no de-obfuscation unit, because every packet it releases remains a valid DNP3 packet for the unmodified master.

IX. Conclusion

In this paper, we presented NOMAD, an in-network traffic obfuscation mechanism against ICS device fingerprinting over plain-text communication. NOMAD uses the syntax of control units to apply a different strategy to each type of operation on one programmable switch: it replaces the CLRT of READ and SELECT with a configured value, holds OPERATE commands inside the switch, and splits the control units of every response into one fixed layout. On a Tofino-1 testbed with 98,816 DNP3 transactions, the CLRT at the master stayed within 10–16 μs of the configured value at the median, and all responses carried 61 bytes and remained conformant to IEEE 1815. The CLRT classifier of Formby et al., trained before deployment, fell to chance in every setting, as it also did under a constant delay, and the same classifier retrained after deployment stayed near its unobfuscated accuracy on our software outstation, whose unobfuscated CLRT barely differs between operations. All transactions completed within 53 ms without retransmission. In future work, we will evaluate NOMAD with physical relays of several vendors, whose wider timing gives the classifier more to learn, and extend the strategies to requests.

Acknowledgments

Appendix A
Security Support Documented by IED Vendors

To estimate how widely DNP3 security is available in deployed devices, we collected public product manuals, data sheets and pages of five IED vendors and recorded, for each device, the DNP3 security it documents. Table III summarizes 55 devices. Of these, 48 document no DNP3 security, 4 document encryption with authentication, 2 document authentication only, and 1 lists an option we could not confirm. All 25 devices of mature or legacy generations document none. Two of the four encrypted entries are PLCs whose security protects EtherNet/IP rather than DNP3, which reaches them only through an add-on module. A feature that a vendor documents may still be unused in a deployment [18]; an analysis of traffic at an Internet exchange found that 75.6% of ICS hosts communicate without encryption [53].

TABLE III
DNP3 security support documented by five IED vendors (device counts): encryption with authentication, authentication only, an unconfirmed option, and none.

Vendor	Devices	Enc.+Auth.	Auth.	Unconf.	None
SEL	15	1	0	0	14
GE	12	0	0	0	12
Siemens	11	1	0	0	10
Schneider	10	0	2	1	7
Rockwell	7	2	0	0	5
Total	55	4	2	1	48

Appendix B
Implementation on Tofino-1

We implement the three modules of Section IV as three $P4_{16}$ programs for the Tofino Native Architecture, compiled with p4c 9.13. The switch has two physical pipes. The classifier runs in the ingress of pipe 0 and hands the requests of timed operations to the holder, which runs in the ingress of pipe 1; the rewriter runs in the egress of pipe 0, through which every packet toward an endpoint leaves. Each pipe has dedicated recirculation ports, which carry the blocker tokens and the held packets between passes. The holder reads the ingress timestamp in units of 256 ns, and the rewriter updates the TCP checksum incrementally. The classifier and the holder each use 11 match/action stages and the rewriter uses all 12 stages of its pipe, and on average the programs occupy 16.5% of the SRAM and 31.3% of the TCAM of pipe 0.

References

[1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the Ukrainian power grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016, tLP: White, 18 March 2016.

[2] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2009.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2012, pp. 332–346.

[5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.

[6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.

[7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN traffic obfuscation at line rate," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2022.

[8] Z. Wang, Q. Li, G. Xie, D. Zhao, K. Li, Z. Fan, L. Ma, and Y. Jiang, "Minos: A lightweight and dynamic defense against traffic analysis in programmable data planes," in *Proc. USENIX Annual Technical Conference (ATC)*, 2025, pp. 399–415. [Online]. Available: https://www.usenix.org/conference/atc25/presentation/wang-zihao

[9] G. Xie, Q. Li, Z. Shi, G. Antichi, Y. Zhu, K. Li, C. Weng, S. Miano, Y. Jiang, and M. Xu, "Defending against traffic analysis attacks with flexible in-network obfuscation," in *Proc. 23rd USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2026, pp. 2043–2063. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/xie-guorui

[10] IEEE, "IEEE standard for electric power systems communications—distributed network protocol (DNP3)," IEEE Std 1815-2012, 2012, iEEE Std 1815-2012.

[11] D. Formby, P. Srinivasan, A. M. Leonard, J. D. Rogers, and R. A. Beyah, "Who's in control of your control system? Device fingerprinting for cyber-physical systems," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[12] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[13] The P4.org Architecture Working Group, "$P4_{16}$ portable switch architecture (PSA), version 1.2," P4.org, Tech. Rep., Dec. 2022. [Online]. Available: https://p4.org/wp-content/uploads/sites/53/p4-spec/docs/PSA-v1.2.html

[14] S. East, J. Butts, M. Papa, and S. Shenoi, "A taxonomy of attacks on the DNP3 protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol. 311)*. Springer, 2009, pp. 67–81.

[15] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.

[16] Schweitzer Engineering Laboratories, *SEL-751A Feeder Protection Relay Instruction Manual*, 2013, pM751A-01-NB, 20130329 printing; settings table, `STIMEO1` select/operate time-out.

[17] Schneider Electric, "PowerLogic ION7550/ION7650 series," Product range page, accessed Oct. 15, 2026. [Online]. Available: https://www.se.com/us/en/product-range/1460-powerlogic-ion7550-ion7650-series/

[18] Cybersecurity and Infrastructure Security Agency, "Barriers to secure OT communication: Why Johnny can't authenticate," Cybersecurity and Infrastructure Security Agency (CISA), Guidance, Feb. 2026. [Online]. Available: https://www.cisa.gov/sites/default/files/2026-02/Barriers-to-Secure-Communication-Why-OT-Johnny-Cant-Authenticate_508_2.pdf

[19] Automatak, "opendnp3: An open-source implementation of the DNP3 (IEEE-1815) protocol stack," https://github.com/dnp3/opendnp3, accessed 2026.

[20] T. Kohno, A. Broido, and K. C. Claffy, "Remote physical device fingerprinting," *IEEE Trans. on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.

[21] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.

[22] J. Hayes and G. Danezis, "K-fingerprinting: A Robust Scalable Website Fingerprinting Technique," in *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, 2016, pp. 1187–1203.

[23] M. S. Rahman, P. Sirinam, N. Mathews, K. G. Gangadhara, and M. Wright, "*Tik-Tok* : The Utility of Packet Timing in Website Fingerprinting Attacks," *Proceedings on Privacy Enhancing Technologies*, vol. 2020, no. 3, pp. 5–24, Jul. 2020.

[24] D. Formby, A. Walid, and R. Beyah, "A Case Study in Power Substation Network Dynamics," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 1, no. 1, pp. 1–24, Jun. 2017.

[25] C. Shen, C. Liu, H. Tan, Z. Wang, D. Xu, and X. Su, "Hybrid-augmented device fingerprinting for intrusion detection in industrial control system networks," *IEEE Wireless Communications*, vol. 25, no. 6, pp. 26–31, 2018.

[26] C. M. Ahmed, M. Ochoa, J. Zhou, and A. Mathur, "Scanning the cycle: Timing-based authentication on PLCs," in *Proc. ACM Asia Conf. on Computer and Communications Security (ASIACCS)*, 2021, pp. 886–900.

[27] O. Pospisil and R. Fujdiak, "Identifying industry devices via time delay in dataflow," in *Proc. 13th Int. Conf. on Communication and Network Security (ICCNS)*, 2023, pp. 173–177.

[28] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for cyber-physical system security: Device physics matters too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, 2018.

[29] L. Babun, H. Aksu, and A. S. Uluagac, "CPS Device-Class Identification via Behavioral Fingerprinting: From Theory to Practice," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2413–2428, 2021.

[30] A. Keliris and M. Maniatakos, "Remote field device fingerprinting using device-specific Modbus information," in *2016 IEEE 59th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, Oct. 2016, pp. 1–4.

[31] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in *Proc. 6th ACM Symp. on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.

[32] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing physical function virtualization to disrupt reconnaissance of power grids' cyber-physical infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.

[33] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of network communication in power grid cyber infrastructure to mislead attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.

[34] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-physical testbed: Case study to evaluate anti-reconnaissance approaches on power grids' cyber-physical infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.

[35] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. European Symp. on Research in Computer Security (ESORICS)*, 2016, pp. 27–46.

[36] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1375–1390.

[37] W. De la Cadena, A. Mitseva, J. Hiller, J. Pennekamp, S. Reuter, J. Filter, T. Engel, K. Wehrle, and A. Panchenko, "TrafficSliver: Fighting Website Fingerprinting Attacks with Traffic Splitting." ACM, Oct. 2020, pp. 1971–1985.

[38] J. Gong and T. Wang, "Zero-delay Lightweight Defenses against Website Fingerprinting," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020, pp. 717–734.

[39] J. Gong, W. Zhang, C. Zhang, and T. Wang, "Surakav: Generating Realistic Traces for a Strong Website Fingerprinting Defense." IEEE, May 2022, pp. 1558–1573.

[40] J. K. Holland and N. Hopper, "RegulaTor: A Straightforward Website Fingerprinting Defense," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 2, pp. 344–362, Mar. 2022.

[41] M. Handley, V. Paxson, and C. Kreibich, "Network intrusion detection: Evasion, traffic normalization, and End-to-End protocol semantics," in *10th USENIX Security Symposium (USENIX Security 01)*. Washington, D.C.: USENIX Association, Aug. 2001. [Online]. Available: https://www.usenix.org/conference/10th-usenix-security-symposium/network-intrusion-detection-evasion-traffic-normalization

[42] M. Smart, G. R. Malan, and F. Jahanian, "Defeating TCP/IP stack fingerprinting," in *9th USENIX Security Symposium (USENIX Security 00)*. Denver, CO: USENIX Association, Aug. 2000. [Online]. Available: https://www.usenix.org/conference/9th-usenix-security-symposium/defeating-tcpip-stack-fingerprinting

[43] A. Askarov, D. Zhang, and A. C. Myers, "Predictive black-box mitigation of timing channels," in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS '10)*. Chicago, IL, USA: ACM, Oct. 2010, pp. 297–307.

[44] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2019, no. 3, pp. 128–148, 2019.

[45] A. Mehta, M. Alzayat, R. De Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive network side-channel mitigation in the cloud," in *Proc. 31st USENIX Security Symposium*, 2022.

[46] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A differentially private network side-channel mitigation system," in *Proc. 33rd USENIX Security Symposium*, 2024.

[47] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, A. Agrawal, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable packet scheduling at line rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.

[48] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating push-in first-out behaviors using strict-priority queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.

[49] N. K. Sharma, C. Zhao, M. Liu, P. G. Kannan, C. Kim, A. Krishnamurthy, and A. Sivaraman, "Programmable Calendar Queues for High-speed Packet Scheduling," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, 2020, pp. 685–699.

[50] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and practical network topology obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.

[51] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial network protocol security enhancement using programmable switches," in *Proc. IEEE Int. Conf. on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.

[52] Z. Hu, H. Lin, Y. Qu, and D. Jin, "I-DETECT: In-network Deep Packet Inspection for Timed Event Correlation in Industrial Control Systems." IEEE, Jun. 2026, pp. 627–640.

[53] G. Barbieri, M. Conti, N. O. Tippenhauer, and F. Turrin, "Assessing the Use of Insecure ICS Protocols via IXP Network Traffic Analysis." IEEE, Jul. 2021, pp. 1–9.